



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №7
Технології розробки програмного забезпечення
«ШАБЛОН «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE
METHOD»»

Виконав:
студент групи ІА-24
Іскандаров Е. Е

Перевірив:
Мягкий М. Ю.

Тема: «ШАБЛОН «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE METHOD»»

Зміст

Короткі теоретичні відомості.....	2
Реалізація шаблону проектування	3
Структура шаблону	5

Короткі теоретичні відомості

Шаблон «Mediator» координує взаємодію між об'єктами через центральний об'єкт, замінюючи прямі зв'язки між ними. Це спрощує структуру та забезпечує керованість системи.

«Facade» надає єдиний уніфікований інтерфейс для доступу до складної підсистеми, приховуючи її внутрішню реалізацію. Це спрощує роботу з підсистемою, зменшує залежності та запобігає «спагеті-коду».

«Bridge» розділяє інтерфейс і його реалізацію, дозволяючи розвивати їх незалежно. Це забезпечує гнучкість при додаванні нових абстракцій і способів їх реалізації.

Шаблон «Template Method» визначає загальну структуру алгоритму в базовому класі, залишаючи реалізацію окремих кроків підкласам. Це забезпечує стандартизацію алгоритму та дозволяє змінювати його окремі частини без впливу на загальну логіку.

Хід роботи:

Реалізація патерну **Template Method** дозволяє визначити шаблон алгоритму в суперкласі та делегувати реалізацію деяких його кроків підкласам. У контексті веб-браузера ми можемо створити базовий клас для обробки HTTP-відповідей, де основний алгоритм залишається незмінним, а специфічні частини (як обробка 404 чи 502/503) реалізуються в підкласах.

```

1 package org.example.template;
2
3 abstract class HttpResponseHandler { 1 usage 1 inheritor
4     public final void processResponse(int statusCode, String content) { no usages
5         logResponse(statusCode, content);
6         if (statusCode >= 200 && statusCode < 300) {
7             handleSuccess(content);
8         } else if (statusCode >= 300 && statusCode < 400) {
9             handleRedirection(content);
10        } else if (statusCode == 404) {
11            handleNotFound();
12        } else if (statusCode == 502 || statusCode == 503) {
13            handleServerError();
14        } else {
15            handleUnknownStatus(statusCode);
16        }
17    }
18
19    private void logResponse(int statusCode, String content) { 1 usage
20        System.out.println("Status Code: " + statusCode);
21        System.out.println("Content: " + content);
22    }
23
24    protected abstract void handleSuccess(String content); 1 usage 1 implementation
25
26    protected abstract void handleRedirection(String content); 1 usage 1 implementation
27
28    protected abstract void handleNotFound(); 1 usage 1 implementation
29
30    protected abstract void handleServerError(); 1 usage 1 implementation
31
32    protected abstract void handleUnknownStatus(int statusCode); 1 usage 1 implementation
33 }
34

```

Рис 1.1 – Код класу HttpResponseHandler

HttpResponseHandler (абстрактний клас):

- Визначає шаблонний метод processResponse, який включає всі кроки обробки HTTP-відповіді.
- Містить абстрактні методи для реалізації специфічної логіки у підкласах.

```

3 class BrowserHttpServletResponseHandler extends HttpServletResponse { 1 usage
4
5     @Override 1 usage
6     protected void handleSuccess(String content) {
7         System.out.println("Rendering HTML content...");
8     }
9
10    @Override 1 usage
11    protected void handleRedirection(String content) {
12        System.out.println("Redirecting to a new URL...");
13    }
14
15    @Override 1 usage
16    protected void handleNotFound() {
17        System.out.println("Displaying 404 Page: Page Not Found.");
18    }
19
20    @Override 1 usage
21    protected void handleServerError() {
22        System.out.println("Displaying 502/503 Page: Server Error.");
23    }
24
25    @Override 1 usage
26    protected void handleUnknownStatus(int statusCode) {
27        System.out.println("Unhandled status code: " + statusCode);
28    }
29 }

```

Рис 1.2 - Код класу BrowserHttpServletResponseHandler

Реалізує специфічну логіку обробки для браузера: успішні відповіді, редіректи, помилки 404, 502/503 тощо.

```

3 public class Main {
4     public static void main(String[] args) {
5         HttpServletResponse handler = new BrowserHttpServletResponseHandler();
6
7         handler.processResponse(statusCode: 200, content: "<html>Success Page</html>");
8         handler.processResponse(statusCode: 301, content: "Redirected Content");
9         handler.processResponse(statusCode: 404, content: "Page Not Found Content");
10        handler.processResponse(statusCode: 503, content: "Server Error Content");
11        handler.processResponse(statusCode: 999, content: "Unknown Response");
12    }
13 }

```

Рис 1.3 – Код класу Main

Використовує конкретну реалізацію BrowserHttpServletResponseHandler для тестування шаблонного методу.

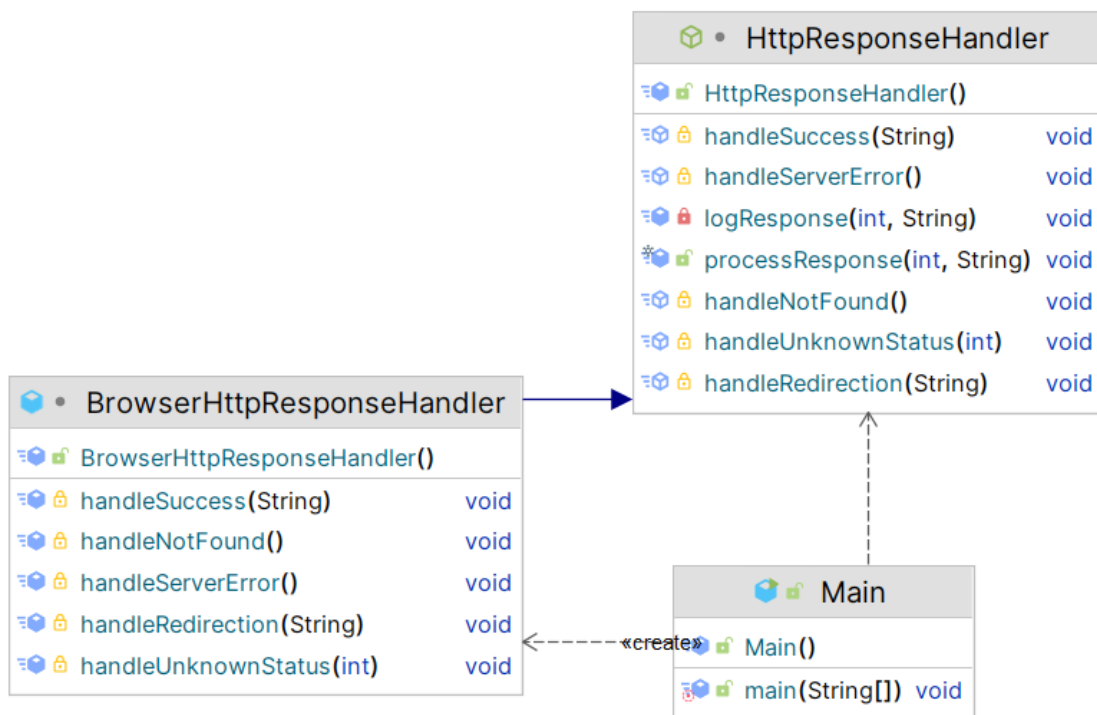


Рис 1.4 – Структура шаблону Template Method

Структура шаблону Template Method включає абстрактний клас, який визначає загальну структуру алгоритму у вигляді шаблонного методу. Цей метод є публічним і визначає послідовність кроків, що складають алгоритм обробки команди. Кроки, які мають змінювану частину, визначені як абстрактні методи, що дозволяє підкласам реалізувати їх відповідно до специфіки команди. Конкретні підкласи успадковують абстрактний клас і надають реалізацію для кожного абстрактного методу, додаючи при цьому свою логіку.

В абстрактному класі також можуть бути реалізовані загальні методи, які використовуються в усіх підкласах. Це забезпечує повторне використання коду та централізоване управління частинами алгоритму, які не змінюються. Завдяки такій структурі, базовий клас контролює порядок виконання, а підкласи реалізують деталі. Всі підкласи слідуєть єдиній схемі обробки, що зменшує складність при розширенні програми.

Структура шаблону дозволяє чітко розмежувати логіку алгоритму і специфіку його виконання, що допомагає спрощувати модифікацію та підтримку коду. Загальні частини алгоритму, які часто використовуються, виносяться в абстрактний клас, що знижує дублювання коду, а специфічні реалізації відокремлюються у підкласах, що забезпечує гнучкість. Це робить програму більш модульною, полегшуючи додавання нових типів команд або зміну існуючих без порушення загальної структури.

Реалізація: https://github.com/Elmir2/trpz_labs

Висновок: У ході виконання даної лабораторної роботи було досліджено структуру, призначення, переваги та недоліки шаблонів проєктування «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE METHOD». Дані патерни мають свої особливі переваги та недоліки, а також використовуються для досягнення різних цілей. Для реалізації частини майбутньої системи було обрано шаблон Template Method, який найкраще підходить для реалізації частини майбутньої системи у даному випадку. Далі було реалізовано обраний патерн, розглянуто його призначення, структуру та досліджено його переваги у використанні.